

EXPERIMENT-7: Servlet controller for web interaction.

Design a controller with servlet that provides interaction with the application developed in experiment 1 and the database created in experiment 5.

Objective : Connect the frontend web app to backend servlet logic and database operations.

Tools : Java Servlets, JDBC, HTML forms, backend routing

Outcome : A servlet-based web controller integrating the shopping app with database CRUD.

Answer:

Handle form requests in servlet methods, use JDBC to update the database, and return the response to the client.

File Name: server.js

Code:

```
// =====  
// Experiment: Web Controller (Servlet-style) with Database CRUD  
// Shopping App + SQLite DB + Node.js (no npm install needed)  
// Run: node server.js   Open: http://localhost:3000  
// Requires Node.js 22.5+ (uses built-in node:sqlite)  
// =====  
  
const http = require('http');  
const { DatabaseSync } = require('node:sqlite');  
  
// ----- DATABASE (like JDBC connection) -----  
const db = new DatabaseSync('shop.db');  
db.exec(`CREATE TABLE IF NOT EXISTS products (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT NOT NULL,  
  price REAL NOT NULL,  
  quantity INTEGER NOT NULL  
)`);  
  
// Insert sample data on first run
```

```

if (db.prepare('SELECT COUNT(*) AS c FROM products').get().c === 0) {
  const ins = db.prepare('INSERT INTO products (name, price, quantity) VALUES (?, ?, ?)');
  ins.run('Laptop', 55000, 5);
  ins.run('Headphones', 1500, 20);
  ins.run('Mouse', 500, 50);
}

```

// ----- HELPERS -----

```

const esc = s => String(s).replace(/([<>"])/g,
  c => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }[c]));

```

```

function readBody(req) {
  return new Promise(resolve => {
    let data = '';
    req.on('data', chunk => (data += chunk));
    req.on('end', () => resolve(Object.fromEntries(new URLSearchParams(data))));
  });
}

```

```

function redirect(res, url) {
  res.writeHead(303, { Location: url });
  res.end();
}

```

// ----- VIEW (HTML frontend) -----

```

function renderPage(products, editItem, search, message) {
  const rows = products.map(p => `
    <tr>
      <td>${p.id}</td>
      <td>${esc(p.name)}</td>
      <td>₹${p.price}</td>
      <td>${p.quantity}</td>
      <td>
        <a href="/?edit=${p.id}">Edit</a>
        <form method="POST" action="/" style="display:inline"
          onsubmit="return confirm('Delete this product?')">

```

```

        <input type="hidden" name="action" value="deleteProduct">
        <input type="hidden" name="id" value="{p.id}">
        <button>Delete</button>
    </form>
</td>
</tr>`).join("");

```

```

const e = editItem;
return `<!DOCTYPE html>
<html><head><meta charset="UTF-8"><title>Shopping App</title>
<style>
    body{font-family:Arial;max-width:800px;margin:30px auto;padding:0 15px}
    table{border-collapse:collapse;width:100%;margin-top:15px}
    th,td{border:1px solid #ccc;padding:8px;text-align:left}
    th{background:#2c3e50;color:#fff}
    input{padding:6px;margin:3px}
    button{padding:6px 12px;cursor:pointer}
    .msg{background:#e8f5e9;padding:8px;margin:10px 0;border-left:4px solid green}
    fieldset{margin-top:15px}
</style></head><body>
<h1>🛒 Shopping App - Product Manager</h1>
${message ? `<div class="msg">${esc(message)}</div>` : ''}

<fieldset>
    <legend>${e ? 'Update Product' : 'Add Product'}</legend>
    <form method="POST" action="/">
        <input type="hidden" name="action" value="${e ? 'updateProduct' : 'addProduct'}">
        ${e ? `<input type="hidden" name="id" value="{e.id}">` : ''}
        <input name="name" placeholder="Name" required value="${e ? esc(e.name) : ''}">
        <input name="price" type="number" step="0.01" min="0" placeholder="Price" required
            value="${e ? e.price : ''}">
        <input name="quantity" type="number" min="0" placeholder="Quantity" required value="${e ?
            e.quantity : ''}">
        <button>${e ? 'Update' : 'Add'}</button>
        ${e ? `<a href="/">Cancel</a>` : ''}
    </form>

```

```
</fieldset>
```

```
<form method="GET" action="/" style="margin-top:15px">
  <input name="search" placeholder="Search by name" value="${esc(search)}">
  <button>Search</button> <a href="/">Show all</a>
</form>
```

```
<table>
  <tr><th>ID</th><th>Name</th><th>Price</th><th>Qty</th><th>Actions</th></tr>
  ${rows || '<tr><td colspan="5">No products found</td></tr>'}
</table>
</body></html>`;
}
```

```
// ----- CONTROLLER -----
```

```
// doGet -> READ (list / search / load item for edit)
```

```
async function doGet(req, res) {
  const url = new URL(req.url, 'http://localhost');
  const search = url.searchParams.get('search') || '';
  const editId = url.searchParams.get('edit');
  const msg = url.searchParams.get('msg') || '';

  const products = db.prepare('SELECT * FROM products WHERE name LIKE ? ORDER BY id')
    .all(`%${search}%`);
  const editItem = editId ? db.prepare('SELECT * FROM products WHERE id = ?').get(editId) : null;

  res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
  res.end(renderPage(products, editItem, search, msg));
}
```

```
// doPost -> CREATE / UPDATE / DELETE, chosen by the "action" parameter
```

```
async function doPost(req, res) {
  const f = await readBody(req);
  const action = f.action;
  let msg = '';
```

```

if (action === 'addProduct') {
  db.prepare('INSERT INTO products (name, price, quantity) VALUES (?, ?, ?)')
    .run(f.name, Number(f.price), parseInt(f.quantity));
  msg = 'Product added successfully';
} else if (action === 'updateProduct') {
  db.prepare('UPDATE products SET name = ?, price = ?, quantity = ? WHERE id = ?')
    .run(f.name, Number(f.price), parseInt(f.quantity), Number(f.id));
  msg = 'Product updated successfully';
} else if (action === 'deleteProduct') {
  db.prepare('DELETE FROM products WHERE id = ?').run(Number(f.id));
  msg = 'Product deleted successfully';
} else {
  msg = 'Unknown action';
}

redirect(res, '/?msg=' + encodeURIComponent(msg)); // Post-Redirect-Get
}

// ----- SERVER (like servlet container: routes request to doGet/doPost) -----
http.createServer(async (req, res) => {
  try {
    if (req.method === 'GET') await doGet(req, res);
    else if (req.method === 'POST') await doPost(req, res);
    else { res.writeHead(405); res.end('Method Not Allowed'); }
  } catch (err) {
    console.error(err);
    res.writeHead(500, { 'Content-Type': 'text/plain' });
    res.end('Server error: ' + err.message);
  }
}).listen(3000, () => console.log('Server running at http://localhost:3000'));

```